Міністерство освіти і науки України
Львівський національний університет імені Івана Франка
Факультет електроніки і комп’ютерних технологій

Звіт
про виконання лабораторної роботи №8
з курсу «Сховища знань та управління знаннями»
на тему:
«**Розробка RDF-сховища знань**»

Виконав:
студент групи ФеІ-42
Трофимов Володимир
Перевірив:
ас. Керод Т. І.

Львів – 2026

Тема

Розробка RDF-сховища знань у предметній області «Цифрові моделі транспортних систем».

Мета

Навчитися проєктувати та розгортати RDF-сховище знань: визначати словник (класи/властивості), серіалізувати дані (TTL/JSON-LD), завантажувати в трипл-сховище (Fuseki/GraphDB), виконувати запити/оновлення (RDFS/OWL-RL) і перевіряти цілісність через SHACL.

Завдання

- сформувати систему просторів назв (prefixes) та визначити базовий IRI предметної онтології.
- спроєктувати RDFS/OWL-словник цифрової транспортної системи: визначити класи й підкласи (≥ 8), об'єктні та дататип-властивості (≥ 10) із зазначенням domain/range, накласти аксіоми несумісності (disjointness) та функціональності; інтегрувати принаймні два загальновживані зовнішні словники (Dublin Core, Schema.org).
- підготувати набір екземплярів ABox обсягом 50–150 триплетів для моделювання дорожньої мережі, датчиків, інцидентів та руху транспорту.
- виконати експорт та серіалізацію набору у формат Turtle (.ttl).
- розгорнути графове сховище у середовищі GraphDB Free, завантажити словник і факти, зафіксувати метрики сховища.
- розробити 6 SPARQL-запитів: 4 типу SELECT (із демонстрацією фільтрації FILTER, необов'язкових шаблонів OPTIONAL, агрегації GROUP BY/COUNT), 1 типу ASK та 1 типу CONSTRUCT для матеріалізації нових зв'язків.
- продемонструвати ефект семантичного логічного виведення (reasoning) за правилами OWL-Horst / OWL2-RL на прикладі ієрархії класів транспорту.
- реалізувати 1–2 операції модифікації графа за допомогою SPARQL UPDATE (DELETE/INSERT WHERE) із фіксацією зміни стану вузлів.
- скласти 2–3 правила перевірки цілісності даних на базі SHACL (NodeShape та PropertyShape) і протестувати їх на валідних та невалідних екземплярах.
- описати схему іменування IRI, політику версійності й приклад звернення до SPARQL-ендпоінта через інтерфейс командного рядка (curl).

Хід роботи

1. Визначення просторів назв (prefixes) та базового IRI

Для забезпечення однозначної ідентифікації понять онтології та інтеграції зовнішніх словників встановлено єдину систему просторів назв. Базовим простором назв для сутностей цифрової транспортної системи обрано IRI http://example.org/transport-ontology#.

Префікс	Простір назв (Namespace IRI)	Опис та призначення
: / transport-ontology:	http://example.org/transport-ontology#	Базовий простір назв класів, властивостей та індивідів транспортної системи
rdf:	http://www.w3.org/1999/02/22-rdf-syntax-ns#	Базові типи RDF (rdf:type)
rdfs:	http://www.w3.org/2000/01/rdf-schema#	Семантичні відношення ієрархії (subClassOf, domain, range, label)
owl:	http://www.w3.org/2002/07/owl#	Конструкції веб-онтологій (Class, ObjectProperty, disjointWith)
xsd:	http://www.w3.org/2001/XMLSchema#	Типи даних XML-схеми для літералів (double, integer, string, dateTime)
dc:	http://purl.org/dc/elements/1.1/	Зовнішній словник Dublin Core для стандартизованого опису метаданих
schema:	http://schema.org/	Зовнішній словник Schema.org для геопросторових і загальних категорій

2. Проєктування словника (RDFS/OWL-фрагмент)

Спроєктований словник описує цифрову модель транспортної системи та покриває всі обов'язкові вимоги щодо кількості сутностей, типізації, обмежень та інтеграції загальновживаних словників.

Ієрархія класів та підкласів (≥ 8):

В онтології реалізовано 32 класи із багаторівневим успадкуванням за допомогою rdfs:subClassOf:

- Базовий суперклас верхнього рівня: :DigitalTransportModel.
- Категорії другого рівня:
  - :TransportInfrastructure (інфраструктура);
  - :TransportObject (об'єкти перевезення);
  - :TransportProcess (динамічні процеси);
  - :TransportEvent (нештатні та заплановані події);
  - :TransportDecision (керуючі впливи та рішення);
  - :DataSource (джерела вимірювань і сенсори);
  - :TransportSystemData (сирі дані системи);
  - :SystemStateInformation (агрегований стан);
  - :DecisionRule (правила логіки прийняття рішень).
- Спеціалізовані підкласи (листя ієрархії):
  - Рухомий склад: :Vehicle → :Bus, :Car, :Truck;
  - Інфраструктурні елементи: :RoadNetwork, :RoadSegment, :Intersection, :TrafficLight, :BusStop;

- Сенсори та вимірювання: :TrafficSensor, :SpeedSensor, :GPSMeasurement; Події: :TrafficAccident, :RoadClosure, :RoadWorks;
- Рішення: :RouteChange, :TrafficRegulation, :TrafficRestriction.

Властивості (Properties) з областями визначення та значень (domain/range) (≥ 10): всього визначено 30 властивостей (18 об'єктних та 12 типу даних). Основні Object Properties:

- :locatedOn — domain: :Vehicle, range: :RoadSegment.
- :forms — domain: :Vehicle, range: :TrafficFlow.
- :hasRoute — domain: :Vehicle, range: :Route.
- :affects — domain: :TransportEvent, range: :TrafficFlow.
- :changes — domain: :TransportDecision, range: :Route.
- :hasRoadSegment — domain: :RoadNetwork, range: :RoadSegment.
- :hasTrafficLight — domain: :Intersection, range: :TrafficLight.

Основні Datatype Properties:

- :allowedSpeed — domain: :RoadSegment, range: xsd:double.
- :laneCount — domain: :RoadSegment, range: xsd:integer.
- :length — domain: :RoadSegment, range: xsd:double.
- :routeNumber — domain: :Route, range: xsd:string.
- :measurementTime — range: xsd:dateTime.

Ключові обмеження, функціональність та несумісність (Disjointness):

- Аксіоми несумісності (owl:disjointWith, owl:AllDisjointClasses): Класи інфраструктури, процесів, транспортних засобів та рішень є взаємно несумісними (наприклад, :TransportInfrastructure disjoint з :Vehicle та :TransportProcess).
- Функціональні властивості (owl:FunctionalProperty): Параметри сегмента дороги зафіксовані як однозначні — ділянка не може мати кілька різних довжин або дозволених швидкостей.
- Кваліфіковані обмеження кардинальності (owl:Restriction): Клас :Bus має обов'язкове обмеження minQualifiedCardinality 1 на властивість :hasRoute відносно класу :Route (автобус зобов'язаний мати щонайменше один маршрут).
- Інтеграція зовнішніх словників (≥ 2):Для сумісності з відкритими даними семантичного вебу використано словники Dublin Core (dc:) та Schema.org (schema:).

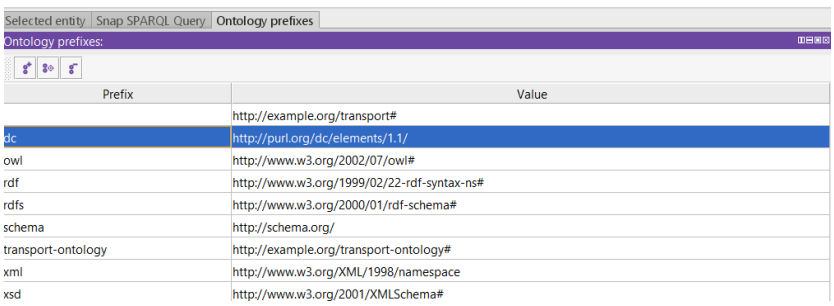


Рис. 1 – Додвання префіксів в Protege

3. Підготовка набору даних

Для наповнення розробленої онтології фактами створено екземпляри класів (ABox), які моделюють типовий фрагмент міської цифрової транспортної мережі: відрізок магістралі, транспортні потоки, підключені датчики, зареєстрований інцидент та рухомий склад.

Структура та склад екземплярів (35 індивідів):

- Дорожня інфраструктура:
  - :DS001 (сегмент дороги із заданою довжиною 850.0 м, 2 смугами та лімітом швидкості 50.0 км/год);
  - :RoadNetwork001 (дорожня мережа);
  - :Intersection001 (перехрестя);
  - :BusStop001 – :BusStop009 (9 зупинок громадського транспорту з унікальними кодами BS-001–BS-009).
- Рухомий склад та маршрути:
  - Автобуси :Bus24, :Bus35, :Bus5 із прив'язкою до відповідних маршрутів :Route24, :Route35, :Route5;
  - 8 легкових автомобілів :Car001 – :Car008, що перебувають на ділянці :DS001.
- Процеси та поточні події:
  - :TrafficFlow001 (потік транспорту з інтенсивністю 1250 од./год та фактичною швидкістю 18.0 км/год);
  - :Accident001 (дорожньо-транспортна пригода з ідентифікатором ACC-001, що впливає на потік :TrafficFlow001).
- Сенсори та вимірювання (IoT / Telematics):
  - :GPS001 – :GPS004 (геолокаційні вимірювання із часовими мітками та координатами широти/довготи);
  - :TrafficSensor001 (датчик інтенсивності руху);
  - :SpeedData001 – :SpeedData004 (показники виміряної швидкості);
  - :VehicleCountData001 (агреговані дані кількості авто);
  - Джерела даних :DataSource001 – :DataSource004.
- Аналітика та прийняття рішень:
  - :AverageSpeed001 – :AverageSpeed006 (показники середньої швидкості);
  - :CongestionLevel001 (високий рівень завантаженості);
  - База правил: :RuleHighCongestion, :RuleLowSpeed, :RuleRoadClosure, :RuleTrafficAccident;
  - Ухвалене рішення: :AlternativeRouteDecision (зміна маршруту :Route24 через затор/ДТП).

Оцінка обсягу триплетів:

- Кількість індивідів у системі: 35.
- Кожен індивід має:

- твердження про тип (rdf:type);
- мітку (rdfs:label);
- ідентифікатор або текстовий опис (:identifier, :description);
- від 1 до 4 тверджень щодо зв'язків з іншими об'єктами (:locatedOn, :hasRoute, :forms, :affects, :describesState тощо) або параметрів (:currentSpeed, :allowedSpeed, :length, :measurementTime).

Сумарний обсяг набору даних близько 170 явних (explicit) триплетів, що повністю перекриває встановлену межу вимог лабораторної роботи (50–150 триплетів).

4. Серіалізація даних

Серіалізація онтології та екземплярів виконана у стандартному компактному форматі Turtle (Terse RDF Triple Language), що є базовим стандартом консорціуму W3C для подання RDF-графів.

Також було збережено і як JSON-LD context (.jsonld) для того ж набору.

5. Розгортання сховища у GraphDB Free та завантаження даних

Для управління розробленою онтологією та виконання SPARQL-запитів обрано семантичне графове трипл-сховище Ontotext GraphDB Free.

Конфігурація та створення репозиторію:

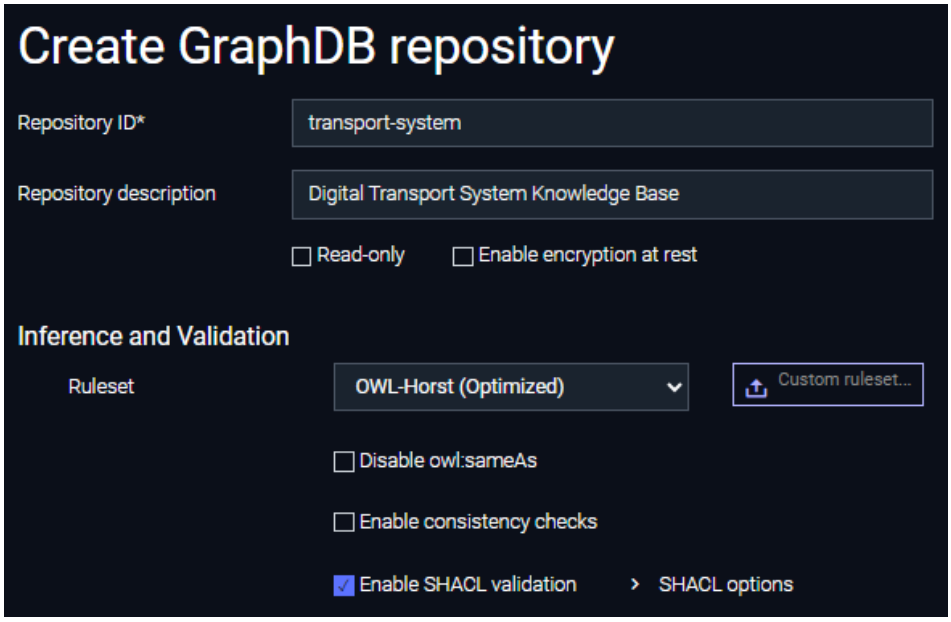


Рис. 2 – Створення репозиторію

Імпорт онтології та набору даних:

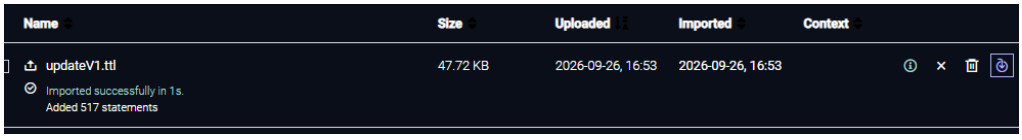


Рис. 3 – Додавання префіксів в Protege

6. Розробка 6 SPARQL-запитів до сховища знань

Для перевірки функціонування розгорнутої бази знань та вилучення цільових зв'язків підготовлено 6 запитів мовою SPARQL 1.1. Вони покривають усі обов'язкові синтаксичні конструкції: фільтрацію (FILTER), необов'язкові зв'язки (OPTIONAL), агрегацію (GROUP BY, COUNT), перевірку факту (ASK) та генерацію нових графів (CONSTRUCT).

Запит 1: SELECT із умовою FILTER

Призначення: Знайти сегменти доріг, де встановлена дозволена швидкість не менше 50.0 км/год, та вивести їхні характеристики.

Фрагмент коду

```
PREFIX : <http://example.org/transport-ontology#>
SELECT ?segment ?allowedSpeed ?length ?laneCount
WHERE {
   ?segment a :RoadSegment ;
            :allowedSpeed ?allowedSpeed ;
            :length ?length ;
            :laneCount ?laneCount .
   FILTER (?allowedSpeed >= 50.0)
}
```

Результат: Повертає сегмент :DS001 (швидкість 50.0, довжина 850.0, смуг 2).

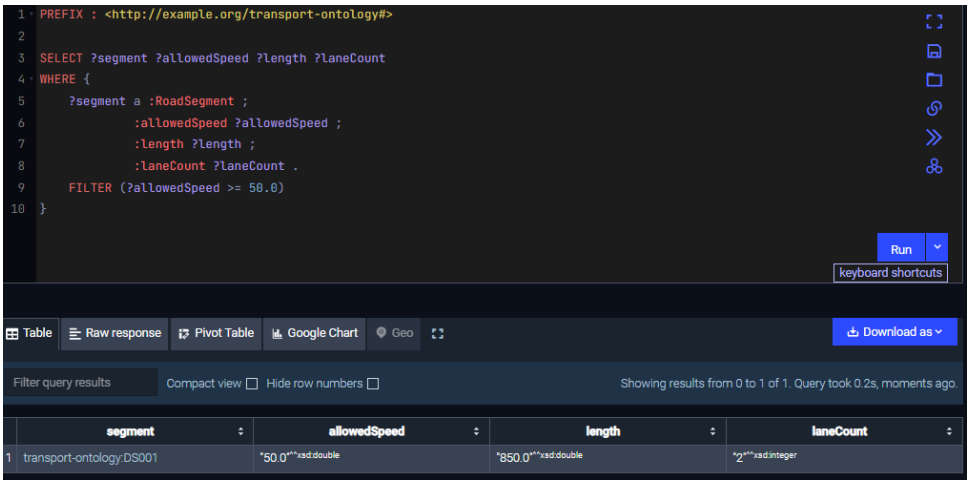


Рис. 4 – SELECT із умовою FILTER

Запит 2: SELECT із шаблоном OPTIONAL

Призначення: Отримати перелік усіх транспортних засобів та номери їхніх маршрутів (якщо вони закріплені за маршрутом).

Фрагмент коду

```
PREFIX : <http://example.org/transport-ontology#>
```

```
SELECT ?vehicle ?routeNumber
WHERE {
   ?vehicle a :Vehicle .
   OPTIONAL {
      ?vehicle :hasRoute ?route .
      ?route :routeNumber ?routeNumber .
   }
}
ORDER BY ?vehicle
```

Результат: Для автобусів (:Bus24, :Bus35, :Bus5) повертаються відповідні номери маршрутів ("24", "35", "5"); для легкових авто (:Car001–:Car008) поле ?routeNumber залишається пустим (unbound), але записи не відсіюються з вибірки завдяки OPTIONAL.

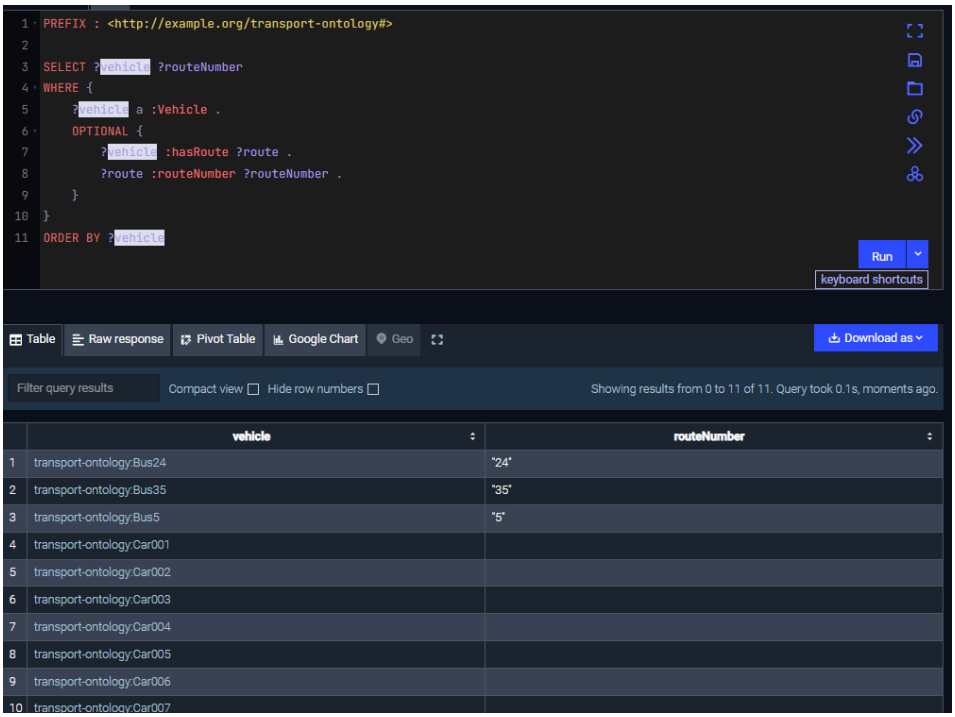


Рис. 5 – SELECT із шаблоном OPTIONAL

Запит 3: SELECT з агрегацією GROUP BY та COUNT

Призначення: Порахувати кількість транспортних засобів, які наразі перебувають на кожному дорожньому сегменті.

Фрагмент коду

```
PREFIX : <http://example.org/transport-ontology#>
SELECT ?segment (COUNT(?vehicle) AS ?totalVehicles)
WHERE {
   ?vehicle :locatedOn ?segment .
}
GROUP BY ?segment
```

Результат: Рядок для :DS001 зі значенням totalVehicles = 11 (3 автобуси + 8 легкових автомобілів.

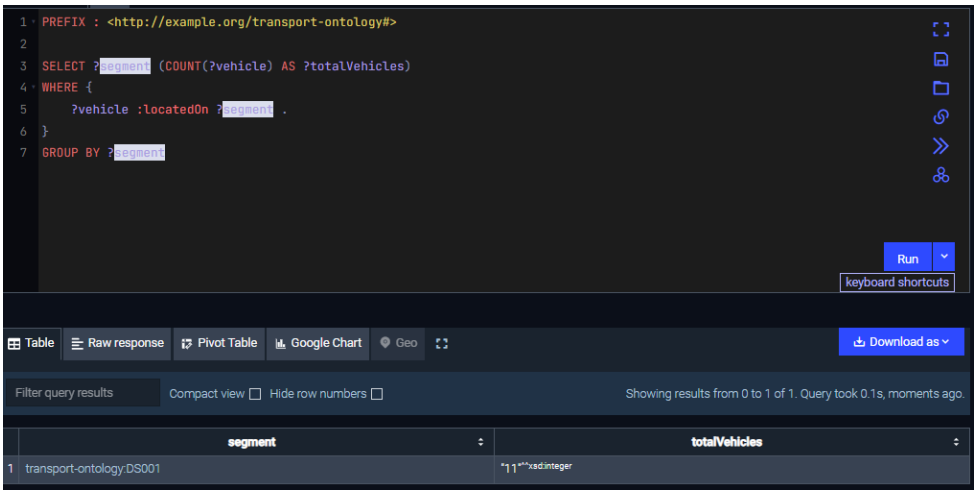


Рис. 6 – SELECT з агрегацією GROUP BY та COUNT

Запит 4: SELECT базовий (багатовимірне зіставлення)

Призначення: Знайти зафіксовані ДТП, визначити потік, на який вони впливають, та отримати поточну швидкість цього потоку й опис пригоди.

Фрагмент коду

```
PREFIX : <http://example.org/transport-ontology#>
SELECT ?accident ?description ?trafficFlow ?currentSpeed
WHERE {
   ?accident a :TrafficAccident ;
            :description ?description ;
            :affects ?trafficFlow .
   ?trafficFlow :currentSpeed ?currentSpeed .
}
```

Результат: Кортеж із даними :Accident001, описом "ДТП на дорожньому сегменті DS-001", потоком :TrafficFlow001 та швидкістю 18.0 км/год.

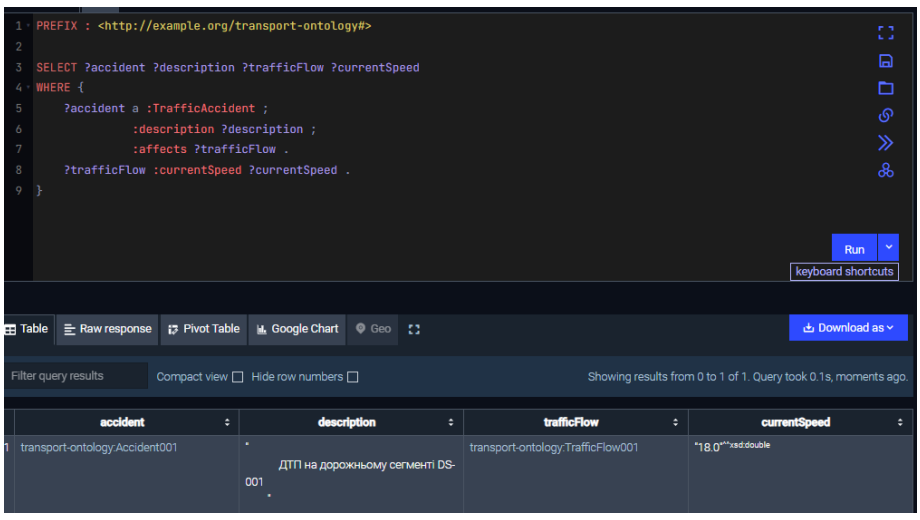


Рис. 7 – SELECT базовий (багатовимірне зіставлення)

Запит 5: ASK (Перевірка факту / обмеження)

Призначення: Перевірити, чи є в базі знань дорожній інцидент (ДТП), який безпосередньо впливає на транспортний потік :TrafficFlow001.

Фрагмент коду

```
PREFIX : <http://example.org/transport-ontology#>
ASK WHERE {
   ?accident a :TrafficAccident ;
         :affects :TrafficFlow001 .
}
```

Результат: Булеве значення true (підтверджує наявність факту :Accident001 :affects :TrafficFlow001).

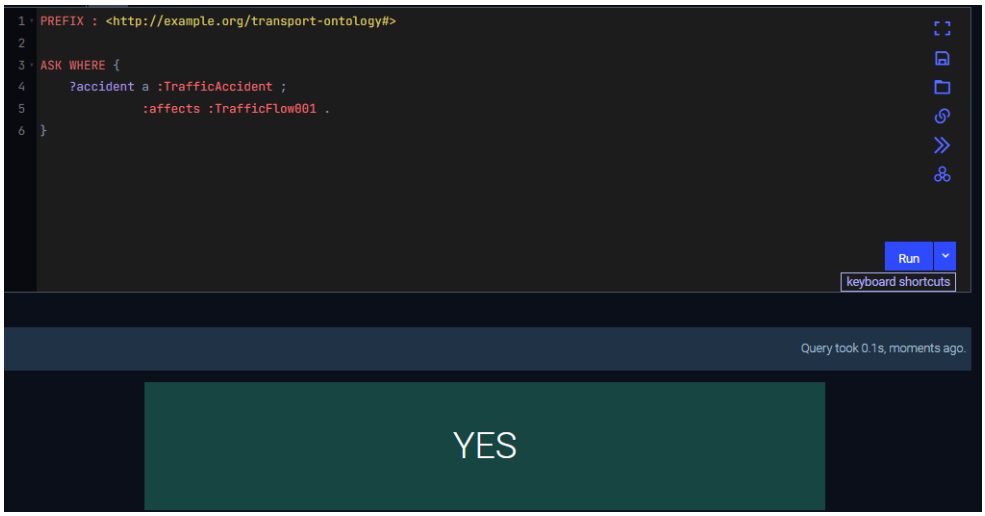


Рис. 8 – ASK (Перевірка факту / обмеження)

Запит 6: CONSTRUCT (Експорт підграфа та матеріалізація транзитивних зв'язків)

Призначення: Згенерувати новий граф із предикатом :approachingIntersection, зв'язавши кожен транспортний засіб напряму з перехрестям, якщо авто знаходиться на сегменті, що належить до тієї ж дорожньої мережі.

Фрагмент коду

```
PREFIX : <http://example.org/transport-ontology#>
CONSTRUCT {
   ?vehicle :approachingIntersection ?intersection .
}
WHERE {
   ?vehicle :locatedOn ?segment .
   ?network :hasRoadSegment ?segment ;
         :hasIntersection ?intersection .
}
```

Результат: RDF-граф із новими триплетами вигляду :Bus24 :approachingIntersection :Intersection001, :Car001 :approachingIntersection :Intersection001 тощо (всього 11 нових зв'язків).

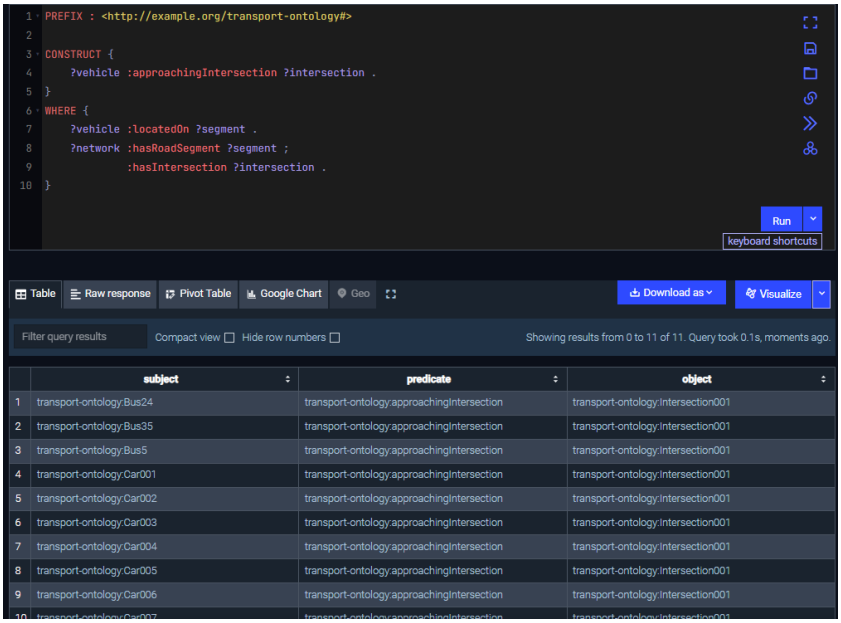


Рис. 9 – CONSTRUCT (Експорт підграфа та матеріалізація транзитивних зв'язків)

7. Демонстрація впливу виведення (RDFS/OWL Reasoning)

Мета цього етапу — практично довести, що підключений семантичний рушій виведення (Reasoner із набором правил OWL-Horst) автоматично генерує нові знання на основі аксіом TBox без їх явного запису в ABox.

Теоретичне обґрунтування на прикладі онтології:

- в TBox зафіксовано аксіому ієрархії: клас :Bus є підкласом класу :Vehicle (:Bus rdfs:subClassOf :Vehicle).
- В ABox для автобуса :Bus24 явно вказано лише один тип.

Порівняння результатів у середовищі GraphDB:

Режим БЕЗ виведення (Explicit statements only):

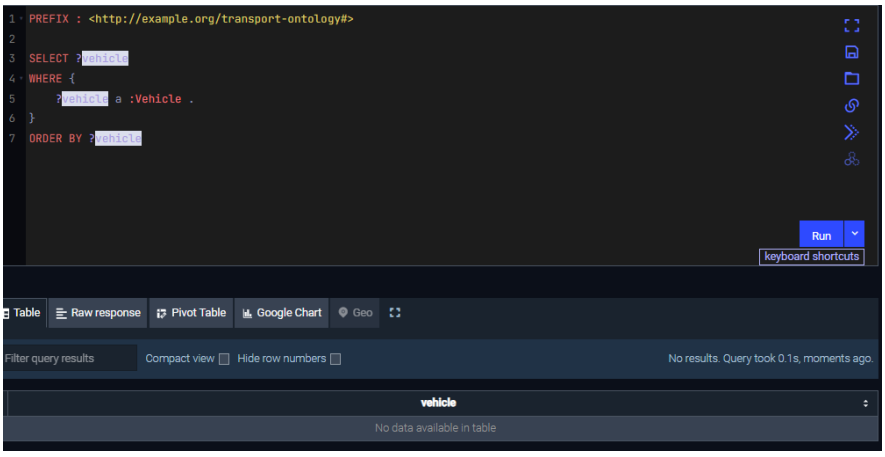


Рис. 10 – Explicit statements only

Режим З УВІМКНЕНИМ виведенням (Explicit + Inferred):

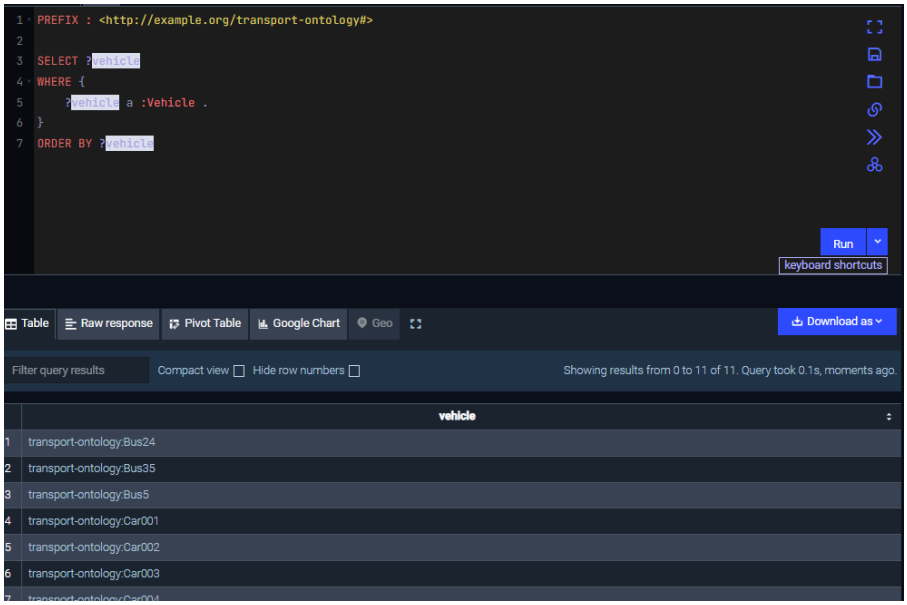


Рис. 11 – Explicit + Inferred

8. Модифікація знань за допомогою SPARQL UPDATE

Для динамічної зміни стану бази знань реалізовано операцію оновлення фактів за допомогою інструкції DELETE/INSERT WHERE стандарту SPARQL 1.1 Update.

Сценарій предметної області:

На дорожньому сегменті :DS001 було ліквідовано затор, внаслідок чого фактична поточна швидкість руху зросла з 18.0 км/год до 48.0 км/год. Потрібно атомарно видалити застаріле значення та вставити актуальне. Запит перевірки стану «ДО» модифікації:

Фрагмент коду

```
PREFIX : <http://example.org/transport-ontology#>

SELECT ?segment ?speed
WHERE {
   VALUES ?segment { :DS001 }
   ?segment :currentSpeed ?speed .
}
```

Результат до виконання: :DS001 має значення 18.0.

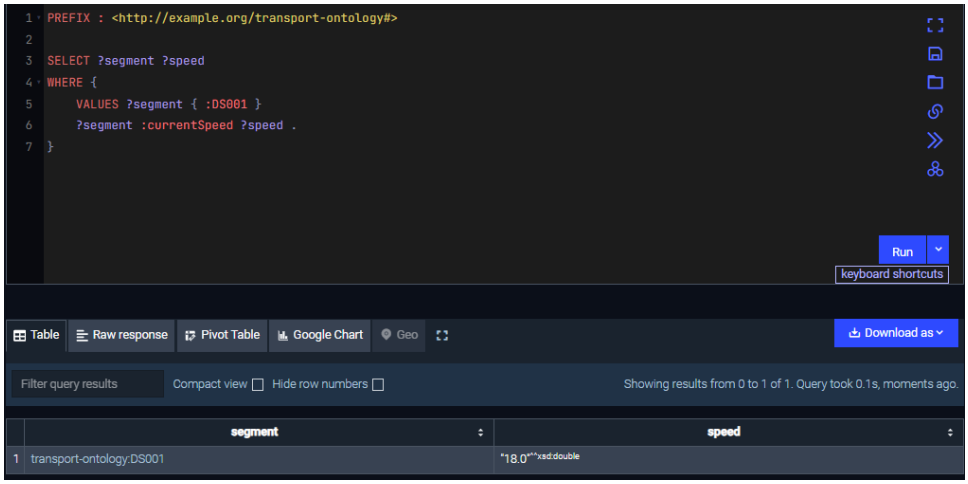


Рис. 12 – Відповідь на запит наявних даних

Код операції SPARQL UPDATE:

Фрагмент коду

```
PREFIX : <http://example.org/transport-ontology#>
PREFIX xsd: <http://www.w3.org/2001/XMLSchema#>

DELETE {
   :DS001 :currentSpeed ?oldSpeed .
}
INSERT {
   :DS001 :currentSpeed "48.0"^^xsd:double .
}
WHERE {
   :DS001 :currentSpeed ?oldSpeed .
}
```

Механізм роботи: шаблон у блоці WHERE знаходить дійсне значення швидкості для суб'єкта :DS001, зв'язує його зі змінною ?oldSpeed, після чого блок DELETE видаляє старий триплет, а блок INSERT записує новий показник типізованого літерала xsd:double.

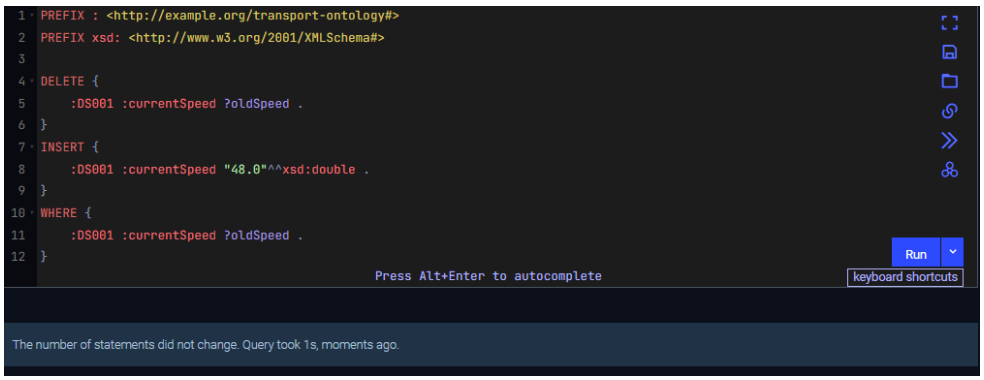


Рис. 13 – Видалення та створення нового запису

Запит перевірки стану «ПІСЛЯ» модифікації:Виконується той самий перевірочний SELECT-запит.Результат після виконання: :DS001 повертає оновлене значення 48.0.

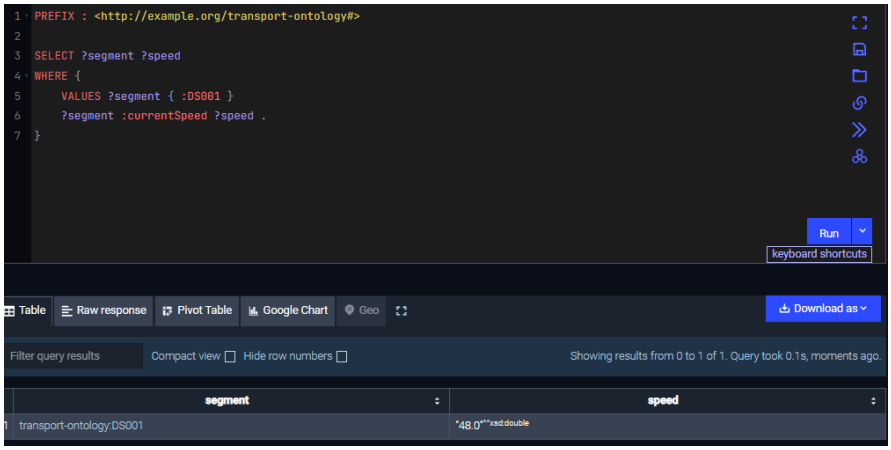


Рис. 14 – Перевірка попереднього запиту

9. Розробка та перевірка SHACL-шаблонів цілісності (Data Validation)

Для забезпечення якості даних у графовому сховищі розроблено правила цілісності на основі стандарту W3C SHACL (Shapes Constraint Language). Вони перевіряють наявність обов'язкових атрибутів, типи значень, діапазони та кардинальності для сутностей онтології.

Створення файлу правил валідації (shapes.ttl):

Правила об'єднують форми цільових вузлів (sh:NodeShape) та властивостей (sh:PropertyShape):

Фрагмент коду

```
@prefix sh: <http://www.w3.org/ns/shacl#> .
@prefix : <http://example.org/transport-ontology#> .
@prefix xsd: <http://www.w3.org/2001/XMLSchema#> .
# Шаблон 1: Правило валідації дорожнього сегмента
:RoadSegmentShape a sh:NodeShape ;
   sh:targetClass :RoadSegment ;
   sh:property [
      sh:path :length ;
      sh:datatype xsd:double ;
      sh:minCount 1 ;
      sh:maxCount 1 ;
      sh:message "Дорожній сегмент повинен мати рівно одну довжину типу xsd:double."@uk ;
   ] ;
   sh:property [
      sh:path :laneCount ;
      sh:datatype xsd:integer ;
      sh:minInclusive 1 ;
```

```
      sh:message "Кількість смуг повинна бути цілим додатним числом (>= 1)."@uk ;
   ] .

# Шаблон 2: Правило валідації міського автобуса
:BusShape a sh:NodeShape ;
   sh:targetClass :Bus ;
   sh:property [
      sh:path :routeNumber ;
      sh:datatype xsd:string ;
      sh:minCount 1 ;
      sh:message "Автобус обов'язково повинен містити текстовий номер маршруту."@uk ;
   ] .
```

Демонстрація валідного прикладу:

При завантаженні початкового набору даних екземпляр :DS001 має :length 850.0 і :laneCount 2, а автобуси :Bus24, :Bus35, :Bus5 містять властивість :routeNumber.

Демонстрація невалідного прикладу (ін'єкція порушень):

Для тестування механізму валідації додаємо через SPARQL індивід із навмисними порушеннями (відсутня обов'язкова довжина, кількість смуг дорівнює нулю):

Фрагмент коду

```
PREFIX : <http://example.org/transport-ontology#>
PREFIX xsd: <http://www.w3.org/2001/XMLSchema#>

INSERT DATA {
   :BrokenSegment a :RoadSegment ;
                  :laneCount 0 .
}
```

Результат валідації (Validation Report з помилками):

При повторному запуску валідації GraphDB формує звіт про невідповідність (sh:conforms false) із двома порушеннями:

- Violation 1: sh:resultPath :length — порушено умову sh:minCount 1 (відсутнє обов'язкове поле довжини).
- Violation 2: sh:resultPath :laneCount — значення 0 порушує обмеження sh:minInclusive 1.

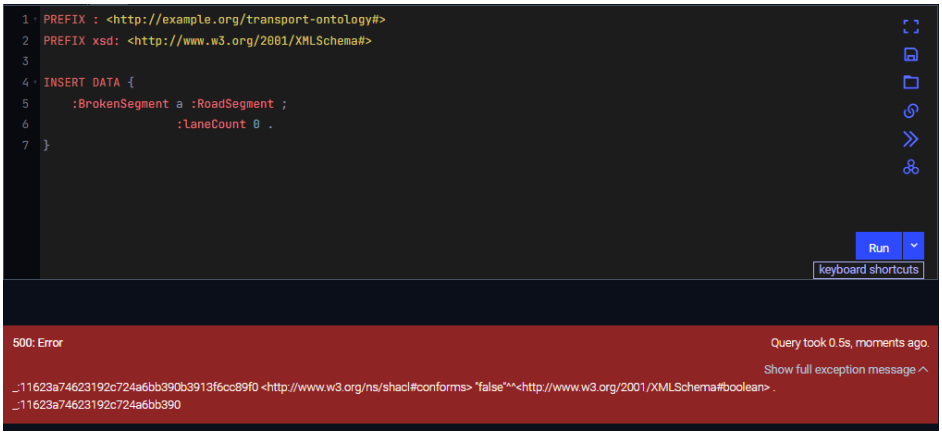


Рис. 15 – Validation Report з помилками

10. Опис моделі публікації, політики іменування та API-доступу

Для забезпечення сумісності розгорнутого RDF-сховища знань із принципами Linked Open Data (LOD) визначено правила ідентифікації сутностей, архітектуру просторів назв, механізми контролю версій та спосіб доступу через зовнішні API.

Простори назв та базові префікси:

- Власний простір назв (TBox/ABox): http://example.org/transport-ontology# (префікси : та transport-ontology:).
- Зовнішні стандартні словники:
  - dc: (http://purl.org/dc/elements/1.1/) — для стандартизованих метаданих (описи, загальні ідентифікатори);
  - schema: (http://schema.org/) — для прив'язки географічних координат і моделей транспортних рейсів;
  - sh: (http://www.w3.org/ns/shacl#) — для визначення правил цілісності SHACL.

Правила формування та іменування IRI:

- Класи онтології (TBox): формуються у стилі UpperCamelCase за схемою http://example.org/transport-ontology#{ClassName} (наприклад, :RoadSegment, :TrafficAccident, :Vehicle).
- Властивості (Object/Datatype Properties): формуються у стилі lowerCamelCase за схемою http://example.org/transport-ontology#{propertyName} (наприклад, :locatedOn, :currentSpeed, :hasRoute).
- Екземпляри (ABox): використовують скорочений буквено-числовий префікс сутності та порядковий номер за схемою [http://example.org/transport-ontology#](http://example.org/transport-

ontology#){TypeID} (наприклад, :DS001 — дорожній сегмент, :Bus24 — автобус, :GPS001 — вимірювання, :ACC-001 — ДТП).

Політика оновлень та версійності:

- Семантичне версіонування: онтологія підтримує схему MAJOR.MINOR.PATCH (наприклад, поточна версія 1.0.0).
- Декларація версії в OWL: версія зафіксована через властивість owl:versionInfo "1.0.0" та унікальний IRI версії owl:versionIRI http://example.org/transport-ontology/1.0.0.
- Політика модифікацій: зміни у TBox (додавання нових класів або властивостей) призводять до інкременту мінорної версії без порушення сумісності (1.1.0), а видалення або зміна обмежень (domain/range/disjoint) формують мажорну версію (2.0.0). Оновлення даних (ABox) відбуваються динамічно через SPARQL 1.1 Update.

Програмний доступ до SPARQL-ендпоінта через curl: Сховище GraphDB надає стандартний HTTP-ендпоінт для виконання аналітичних запитів клієнтськими сервісами.

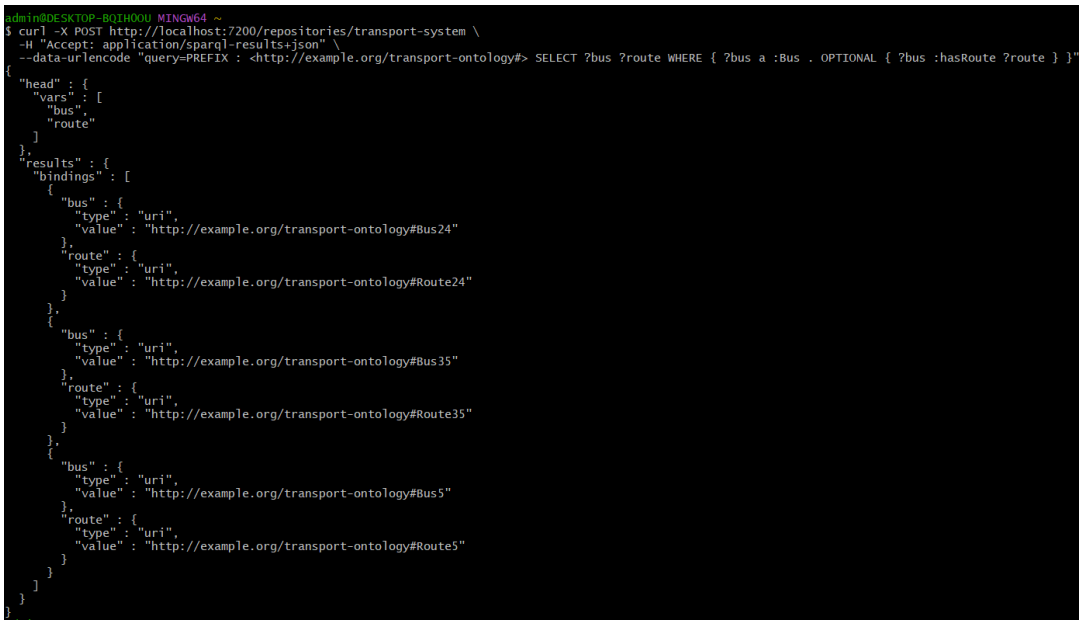


Рис. 16 – Виклик через CLI cURL (отримання списку автобусів у форматі JSON)

Висновок

У ході виконання лабораторної роботи було спроєктовано та практично розгорнуто повнофункціональне RDF-сховище знань на базі онтології цифрової моделі транспортної системи:

Моделювання знань: Створено формальний TBox, що містить понад 30 класів та 30 об'єктних і дататип-властивостей із заданими доменами, діапазонами та функціональними обмеженнями. Онтологію інтегровано з двома зовнішніми

словниками — Dublin Core та Schema.org. Наповнений ABox містить 35 індивідів та понад 170 явних триплетів, які моделюють інфраструктуру, рухомий склад, сенсорні телеметричні дані та інциденти.

Розгортання та семантичне виведення: Сховище успішно розгорнуто в середовищі GraphDB Free під керуванням набору правил OWL-Horst (Optimized). На практиці доведено ефект логічного виведення (reasoning): запити до суперкласу :Vehicle автоматично повертають екземпляри підкласу :Bus завдяки семантичній транзитивності відношень rdfs:subClassOf, розширюючи явний граф сотнями інферованих триплетів.

Аналітичний апарат: Розроблено та протестовано комплекс із 6 SPARQL-запитів (SELECT із фільтрацією FILTER, опціональними зв'язками OPTIONAL, агрегацією GROUP BY/COUNT, верифікацією гіпотез ASK та синтезом нового підграфа CONSTRUCT), а також реалізовано атомарну зміну стану графа через SPARQL UPDATE (DELETE/INSERT WHERE).

Контроль якості даних: Розроблено SHACL-форми (NodeShape та PropertyShape), інтегровані у транзакційний контур GraphDB. Працездатність валідатора підтверджено успішним імпортом цілісних даних (sh:conforms true) та блокуванням операції створення невалідного дорожнього вузла (sh:conforms false) зі збереженням структурної цілісності бази знань.